

Technical Design Document

Banking Application Architecture

File: BankingApp/README.md

Delphi Banking Application

This is a sample Delphi banking application covering major user features:

Features

- User Login
- Dashboard Overview
- Account Details
- Transaction History
- Payments
- Loan Information
- Admin Panel

Structure

- `src/Main.dpr`: Application entry point
- `src/forms/`: UI forms and logic
- `src/modules/`: Data models and business logic
- `src/types/Types.pas`: Common types

How to Run

Open `Main.dpr` in Delphi IDE and run the project.

File: BankingApp/.github/prompts/RevEng.prompt.md

agent: agent

Prompt 1: You are an expert software developer and technical writer. Your task is to perform reverse engineering on a given codebase and generate comprehensive documentation, including technical design documents, user stories, SQA test cases, and a summary report with statistics. Follow the detailed instructions provided in the associated instruction files one by one to ensure accuracy and completeness.

.github/instructions/TDD.instructions.md

.github/instructions/UserSt

File: BankingApp/.github/instructions/TestCases.instructions.md

applyTo:

Goal1: Do the Reverse Engineering the files available in the code base exactly and create the SQA test cases with detailed explanation.

-**Steps**: Detailed steps to execute each test case

-**Preconditions**: Any preconditions that need to be met before executing the test case

-**Test Data**: Specific data required for testing

-**Expected Results**: Clear expected outcomes for each test case

-**Postconditions**: Any conditions that should be met after test ca

File: BankingApp/.github/instructions/RevEng.instructions.md

applyTo: '''

Goal1: Create one HTML file as BankingAppRevEng.html which contains links to all the generated documents and a summary of the reverse engineering process and statistics information.

File: BankingApp/.github/instructions/Statistics.instructions.md

applyTo: '''

Goal1: Provide below statistics about the code base:

- **File Count**: Total number of files in the code base
- **Line Count**: Total number of lines of code across all files
- **Language Breakdown**: Breakdown of files and lines of code by programming language
- **Function/Method Count**: Total number of functions/methods defined in the code base
- **Class Count**: Total number of classes defined in the code base
- **Comment Density**: Ratio of comment line

File: BankingApp/.github/instructions/UserStories.instructions.md

applyTo: '''

Goal1: Do the Reverse Engineering the files available in the code base exactly and create the user stories in detial explanation and cover below points:

- **User Stories**: Detailed user stories for each functionality (login, account management, transactions, payments, loans, admin)
- **Acceptance Criteria**: Clear acceptance criteria for each user story
- **Priority**: Prioritization of user stories based on business needs
- **Dependencies**: Identification of

File: BankingApp/.github/instructions/TDD.instructions.md

applyTo: '''

****Goal1:**** Do the Reverse Engineering the files available in the code base exactly and explain below points in more deeper:

- ****UI Elements & Purpose:**** Detailed descriptions of each form's UI elements and their purpose
- ****User Flow:**** Step-by-step flow for user actions (login, account management, transactions, payments, loans, admin)
- ****Backend Logic:**** Backend logic for each operation (including method details and data flow)
- ****Database Schema:**** Database schema a

File: BankingApp/src/Main.dpr

```
program Main;

uses
  Forms,
  LoginForm in 'forms\LoginForm.pas',
  DashboardForm in 'forms\DashboardForm.pas',
  AccountForm in 'forms\AccountForm.pas',
  TransactionForm in 'forms\TransactionForm.pas',
  PaymentForm in 'forms\PaymentForm.pas',
  LoanForm in 'forms\LoanForm.pas',
  AdminForm in 'forms\AdminForm.pas';

{$R *.res}

begin
  Application.Initialize;
  Application.CreateForm(TLoginForm, LoginForm);
  Application.Run;
end.
```

File: BankingApp/src/types/Types.pas

```
unit Types;

interface

type
  TStatus = (Active, Inactive, Suspended);

implementation

end.
```

File: BankingApp/src/forms/AccountForm.pas

```
unit AccountForm;

interface

uses
```

Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
FireDAC.Stan.Async, FireDAC.DApt;

type

TAccountForm = class(TForm)

lblAccountInfo: TLabel;

lblBalance: TLabel;

lstTransactions: TListBox;

btnDeposit: TButton;

btnWithdraw: TButton;

btnBack: TButton;

edtAmount: TEdit;

lblAmount: TLabel;

FDConnection: TFDConnection;

FDQuery: TFDQuery;

procedure btnBackClick(Sender: TObject);

procedure btnDepositClick(Sender: TO

File: BankingApp/src/forms/LoginForm.dfm

object LoginForm: TLoginForm

Caption = 'Login'

Width = 300

Height = 200

object lblUsername: TLabel

Left = 30

Top = 40

Caption = 'Username:'

end

object edtUsername: TEdit

Left = 110

Top = 40

Width = 150

end

object lblPassword: TLabel

Left = 30

Top = 80

Caption = 'Password:'

end

object edtPassword: TEdit

Left = 110

Top = 80

Width = 150

```
PasswordChar = '*'
end
object btnLogin: TButton
Left = 110
Top = 120
Width =

### File: BankingApp/src/forms/TransactionForm.dfm

object TransactionForm: TTransactionForm
Caption = 'Transactions'
Width = 400
Height = 350
object lblTransactions: TLabel
Left = 20
Top = 20
Caption = 'Transactions'
end
object lstTransactions: TListBox
Left = 20
Top = 50
Width = 350
Height = 180
end
object btnBack: TButton
Left = 20
Top = 310
Width = 80
Caption = 'Back'
OnClick = btnBackClick
end
object edtSearch: TEdit
Left = 20
Top = 240
Width = 200
Hint = 'Se

### File: BankingApp/src/forms/DashboardForm.dfm

object DashboardForm: TDashboardForm
Caption = 'Dashboard'
Width = 400
Height = 300
```

```
object lblWelcome: TLabel
Left = 20
Top = 20
Caption = 'Welcome!'
end
object btnAccount: TButton
Left = 20
Top = 60
Width = 120
Caption = 'Account Details'
OnClick = btnAccountClick
end
object btnTransactions: TButton
Left = 160
Top = 60
Width = 120
Caption = 'Transactions'
OnClick = btnTransactionsClick
end
object btnPayments: TButton
Le
```

File: BankingApp/src/forms/AdminForm.dfm

```
object AdminForm: TAdminForm
Caption = 'Admin Panel'
Width = 400
Height = 350
object lblAdmin: TLabel
Left = 20
Top = 20
Caption = 'Admin Panel'
end
object lblUser: TLabel
Left = 20
Top = 60
Caption = 'Username:'
end
object edtUser: TEdit
Left = 110
Top = 60
```

```

Width = 150
end
object btnAddUser: TButton
Left = 270
Top = 60
Width = 100
Caption = 'Add User'
OnClick = btnAddUserClick
end
object btnRemoveUser: TButton
L

```

```

### File: BankingApp/src/forms/LoginForm.pas

```

```

unit LoginForm;

interface

uses

Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
FireDAC.Stan.Async, FireDAC.DApt;

type
TLoginForm = class(TForm)
edtUsername: TEdit;
edtPassword: TEdit;
btnLogin: TButton;
lblUsername: TLabel;
lblPassword: TLabel;
FDConnection: TFDConnection;
FDQuery: TFDQuery;
procedure btnLoginClick(Sender: TObject);
private
procedure ConnectToOracle;
public
{ Public declarations }
end;

var
LoginForm: TLoginForm;
i

```

```

### File: BankingApp/src/forms/LoanForm.dfm

```

```

object LoanForm: TLoanForm
Caption = 'Loan Information'

```

```
Width = 400
Height = 350
object lblLoanInfo: TLabel
Left = 20
Top = 20
Caption = 'Loan Info'
end
object lblAmount: TLabel
Left = 20
Top = 50
Caption = 'Amount:'
end
object edtAmount: TEdit
Left = 90
Top = 50
Width = 100
end
object lblInterest: TLabel
Left = 20
Top = 90
Caption = 'Interest Rate:'
end
object edtInterest: TEdit
Left = 110
Top = 90
Width = 80
```

```
### File: BankingApp/src/forms/PaymentForm.dfm
```

```
object PaymentForm: TPaymentForm
Caption = 'Payments'
Width = 400
Height = 350
object lblPayments: TLabel
Left = 20
Top = 20
Caption = 'Payments'
end
object lblPayee: TLabel
Left = 20
Top = 50
```

```

Caption = 'Payee:'
end
object edtPayee: TEdit
Left = 90
Top = 50
Width = 150
end
object lblAmount: TLabel
Left = 20
Top = 90
Caption = 'Amount:'
end
object edtAmount: TEdit
Left = 90
Top = 90
Width = 100
end
object btnM

```

```

### File: BankingApp/src/forms/AdminForm.pas

```

```

unit AdminForm;

```

```

interface

```

```

uses

```

```

Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
FireDAC.Stan.Async, FireDAC.DApt;

```

```

type

```

```

TAdminForm = class(TForm)

```

```

  lblAdmin: TLabel;

```

```

  lblUser: TLabel;

```

```

  edtUser: TEdit;

```

```

  btnAddUser: TButton;

```

```

  btnRemoveUser: TButton;

```

```

  lstUsers: TListBox;

```

```

  btnBack: TButton;

```

```

  FDConnection: TFDConnection;

```

```

  FDQuery: TFDQuery;

```

```

  procedure btnBackClick(Sender: TObject);

```

```

  procedure btnAddUserClick(Sender: TObject);

```

```

  procedure btnRemoveUserClick(Se

```

```

### File: BankingApp/src/forms/LoanForm.pas

```

```

unit LoanForm;

interface

uses
  Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
  FireDAC.Stan.Async, FireDAC.DApt;

type
  TLoanForm = class(TForm)
    lblLoanInfo: TLabel;
    lblAmount: TLabel;
    edtAmount: TEdit;
    lblInterest: TLabel;
    edtInterest: TEdit;
    btnApplyLoan: TButton;
    lstLoans: TListBox;
    btnBack: TButton;
    procedure btnBackClick(Sender: TObject);
    procedure btnApplyLoanClick(Sender: TObject);
    procedure btnSearchClick(Sender: TObject);
  procedure
    ### File: BankingApp/src/forms/AccountForm.dfm

    object AccountForm: TAccountForm
      Caption = 'Account Details'
      Width = 400
      Height = 350
      object lblAccountInfo: TLabel
        Left = 20
        Top = 20
        Caption = 'Account Info'
      end
      object lblBalance: TLabel
        Left = 20
        Top = 50
        Caption = 'Balance:'
      end
      object lstTransactions: TListBox
        Left = 20
        Top = 80
        Width = 350

```

```
Height = 100
end
object lblAmount: TLabel
Left = 20
Top = 190
Caption = 'Amount:'
end
object edtAmount: TEdit
Left = 90
```

```
### File: BankingApp/src/forms/TransactionForm.pas
```

```
unit TransactionForm;

interface

uses
Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
FireDAC.Stan.Async, FireDAC.DApt;

type
TTransactionForm = class(TForm)
lblTransactions: TLabel;
lstTransactions: TListBox;
btnBack: TButton;
FDConnection: TFDConnection;
FDQuery: TFDQuery;
procedure btnBackClick(Sender: TObject);
private
FTransactions: TStringList;
procedure LoadTransactions;
procedure ConnectToOracle;
procedure SearchTransactions(c
```

```
### File: BankingApp/src/forms/PaymentForm.pas
```

```
unit PaymentForm;

interface

uses
Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
FireDAC.Stan.Async, FireDAC.DApt;

type
TPaymentForm = class(TForm)
lblPayments: TLabel;
```

```

lblPayee: TLabel;
edtPayee: TEdit;
lblAmount: TLabel;
edtAmount: TEdit;
btnMakePayment: TButton;
lstPayments: TListBox;
btnBack: TButton;
FDConnection: TFDConnection;
FDQuery: TFDQuery;
procedure btnBackClick(Sender: TObject);
procedure btnMakePaymentClick(Sender: TObject)

### File: BankingApp/src/forms/DashboardForm.pas

unit DashboardForm;

interface

uses

Forms, StdCtrls, Controls, Classes, FireDAC.Comp.Client, FireDAC.Stan.Def,
FireDAC.Stan.Async, FireDAC.DApt;

type
TDashboardForm = class(TForm)
lblWelcome: TLabel;
btnAccount: TButton;
btnTransactions: TButton;
btnPayments: TButton;
btnLoans: TButton;
btnAdmin: TButton;
procedure btnAccountClick(Sender: TObject);
procedure btnTransactionsClick(Sender: TObject);
procedure btnPaymentsClick(Sender: TObject);
procedure

### File: BankingApp/src/modules/TransactionModule.pas

unit TransactionModule;

interface

type
TTransaction = record
TransactionID: string;
AccountNumber: string;
Amount: Double;
TransactionType: string;

```

```
Date: TDateTime;  
end;  
  
implementation  
  
end.
```

```
### File: BankingApp/src/modules/LoanModule.pas
```

```
unit LoanModule;  
  
interface  
  
type  
TLoan = record  
LoanID: string;  
AccountNumber: string;  
Amount: Double;  
InterestRate: Double;  
StartDate: TDateTime;  
EndDate: TDateTime;  
end;  
  
implementation  
  
end.
```

```
### File: BankingApp/src/modules/UserModule.pas
```

```
unit UserModule;  
  
interface  
  
type  
TUser = record  
UserID: string;  
Username: string;  
Password: string;  
FullName: string;  
Email: string;  
IsAdmin: Boolean;  
end;  
  
implementation  
  
end.
```

```
### File: BankingApp/src/modules/AccountModule.pas
```

```
unit AccountModule;
```

interface

type

TAccount = record

AccountNumber: string;

Balance: Double;

AccountType: string;

end;

implementation

end.

File: BankingApp/src/modules/PaymentModule.pas

unit PaymentModule;

interface

type

TPayment = record

PaymentID: string;

AccountNumber: string;

Amount: Double;

Payee: string;

Date: TDateTime;

end;

implementation

end.